Лабораторная работа 1. Инференс и развёртывание языковой модели: от одного запроса к серверу под нагрузкой

Главный вопрос. Из чего складывается время ответа языковой модели и как оно меняется, когда пользователей становится много?
Что получится. Собственный сервер модели с OpenAI-совместимым API; клиент-измеритель в каркасе ЛР0; три опыта: длина запроса и время до первого токена, длина ответа и скорость генерации, число одновременных пользователей и пропускная способность; расчёт памяти KV-кеша; паспорт сервера, который служит моделью во всех следующих работах.
Среда. GPU T4 в Colab или Kaggle и vLLM либо CPU и llama.cpp (или Ollama); Python-клиент `openai`; каркас ЛР0 и MLflow.

1. Зачем эта работа

Пользователь ассистента ждёт дважды. Сначала — пока появится первое слово ответа. Потом — пока «допечатается» весь текст. У этих ожиданий разные причины. Прежде чем выдать первый токен, модель обрабатывает весь запрос целиком; затем она генерирует ответ по одному токену за шаг. В RAG-системе из ЛР2 запрос с тремя документами занимает около двух тысяч токенов, а агент из ЛР4 в каждом вызове передаёт описания 24 инструментов — около четырёх тысяч. Для них время до первого токена важнее скорости печати.

Когда пользователей много, сервер обрабатывает их запросы вместе. Суммарно он выдаёт больше токенов в секунду, но каждому пользователю приходится ждать дольше, и на какой-то нагрузке ожидание становится неприемлемым. Где эта граница на вашем оборудовании, вы и выясните.

В этой работе вы развернёте собственный сервер модели, измерите, как время ответа зависит от длины запроса, длины ответа и числа одновременных запросов, и объясните результат устройством генерации. Этот сервер станет моделью для ЛР2–ЛР8, а измеренные числа помогут заранее оценить длительность их серий.

2. Как устроено

2.1. Prefill и decode

Модель работает с токенами — кусками слов из своего словаря; если токены и устройство трансформера для вас новые, прочитайте главы 1.4–1.6 курса Hugging Face [1]. Генерация ответа состоит из двух этапов [2, 3]:

- **prefill** — один проход по всем токенам запроса сразу. Токены обрабатываются параллельно, и этот этап упирается в вычислительную мощность ускорителя. Его результат — первый токен ответа;
- **decode** — затем модель выдаёт по одному токену за шаг. На каждом шаге ей нужно прочитать из памяти все свои веса, а вычислений для одного токена мало, поэтому этот этап упирается в скорость памяти.

Оценка на пальцах. У модели Qwen3-4B-Instruct-2507 около 4 млрд параметров; в формате FP16 это 8 ГБ весов. Память T4 передаёт до 300 ГБ/с [4]. Значит, один шаг decode для одного пользователя не может быть быстрее $8/300 \approx 0{,}027$ с, то есть примерно 37 токенов в секунду. Настоящая скорость будет ниже: на каждом шаге читается ещё и KV-кеш.

2.2. KV-кеш

Чтобы на каждом шаге не пересчитывать весь текст заново, модель хранит для каждого прошлого токена ключи и значения механизма внимания во всех слоях — **KV-кеш** [5]. Его размер растёт с длиной текста:

$$M_{\text{KV}} = 2 \cdot n_{\text{слоёв}} \cdot n_{\text{KV}} \cdot d_{\text{головы}} \cdot b \cdot L.$$

Здесь n_{KV} — число голов ключей и значений (у моделей с grouped-query attention оно меньше числа голов запросов), $d_{\text{головы}}$ — размерность головы, b — байт на число, L — число токенов. У Qwen3-4B-Instruct-2507 36 слоёв, 8 голов KV размерности 128 [6], и в FP16 на один токен уходит $2 \cdot 36 \cdot 8 \cdot 128 \cdot 2 = 147\,456$ байт, то есть 144 КБ. Контекст в 32 тыс. токенов занимает 4,5 ГБ — больше половины того, что остаётся на T4 после загрузки весов. KV-кеш ограничивает и длину контекста, и число пользователей, которых сервер обслуживает одновременно.

У гибридных моделей часть слоёв устроена иначе. Например, у Qwen3.5 кеш ключей и значений есть только у каждого четвёртого слоя. Прежде чем считать, проверьте конфигурацию своей модели.

2.3. Что измеряется

Клиент отправляет запрос в режиме потоковой передачи (streaming) и отмечает время: t_0 — перед отправкой, t_1 — когда пришёл первый фрагмент текста, t_N — когда пришёл последний. Число токенов сообщает сам сервер в поле `usage`.

- **TTFT** (time to first token) $= t_1 - t_0$ — время до первого токена: очередь, prefill и первый шаг.
- **TPOT** (time per output token) $= (t_N - t_1)/(N-1)$ — среднее время одного следующего токена, где N — число токенов ответа.
- **Полное время ответа** $= t_N - t_0 \approx \text{TTFT} + (N-1) \cdot \text{TPOT}$.
- **Пропускная способность** — сколько выходных токенов в секунду сервер выдаёт всем пользователям вместе.
- **Goodput** — сколько запросов в секунду сервер обслуживает, укладываясь в заданные пределы TTFT и TPOT [7].

Рис. 1 показывает эти величины на временной шкале.

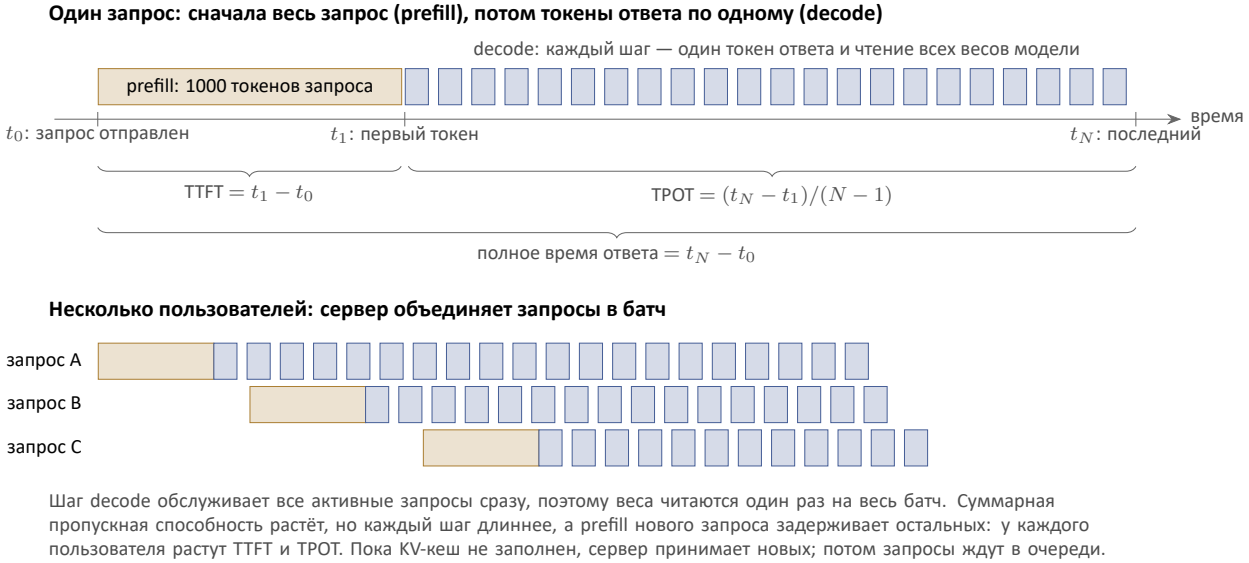


Рис. 1. Один запрос на временной шкале и несколько запросов в одном батче

2.4. Сервер

Сервер с OpenAI-совместимым API принимает те же запросы, что и API OpenAI, поэтому клиентский код не зависит от того, где работает модель [8]. В работе годится любой из трёх вариантов:

- **vLLM** — для GPU. Объединяет запросы в батч на лету (continuous batching) и хранит KV-кеш блоками, почти без потерь памяти (PagedAttention) [9, 10];

- **llama.cpp** (`llama-server`) — для CPU и небольших GPU; работает с квантованными весами в формате GGUF и обслуживает несколько запросов параллельно в «слотах» [11];
- **Ollama** — удобная оболочка над llama.cpp [12].

У серверов есть умолчания, которые влияют на результат:

- **Длина контекста.** По умолчанию vLLM и llama.cpp берут максимум модели — у Qwen3-4B-Instruct-2507 это 262 тыс. токенов, и на T4 vLLM просто не запустится. Задайте предел явно. Ollama на GPU меньше 24 ГБ, наоборот, по умолчанию ставит всего 4 тыс., а агенту из ЛР4 нужно больше. Через OpenAI-совместимый API это не исправить: задайте переменную `OLLAMA_CONTEXT_LENGTH` при запуске `ollama serve` или параметр `num_ctx` в Modelfile [12].
- **Параметры генерации.** vLLM по умолчанию подставляет параметры из `generation_config.json` модели — у этой модели температура 0,7. Всегда передавайте температуру и остальные параметры в запросе явно.
- **Кеширование префикса.** Если начало запроса совпадает с уже обработанным, сервер берёт готовый KV-кеш и пропускает часть prefill. У vLLM и llama.cpp это включено по умолчанию. Для агентов это большой выигрыш, а для замеров TTFT — искажение.
- **Повторяемость.** Даже при температуре 0 ответ может отличаться от запуска к запуску: запросы по-разному группируются в батчи, и вычисления с плавающей точкой дают немного разные числа [13]. На T4 режим, который это исключает, недоступен.

2.5. Нагрузка

При нескольких одновременных запросах один шаг decode обслуживает их все: веса читаются из памяти один раз на весь батч. Поэтому суммарная пропускная способность растёт почти линейно, пока ускоритель не загружен полностью. Но каждый шаг становится длиннее, а prefill нового запроса задерживает остальных, поэтому TTFT и TPOT у каждого пользователя растут. Когда KV-кеш заполнен, новые запросы ждут в очереди, и TTFT растёт скачком. Задача — найти самую большую нагрузку, при которой пределы ещё выполняются.

3. Что сделать

3.1. Сервер и модель

Выберите профиль и запишите его в паспорт сервера.

	GPU	CPU
Сервер	vLLM	llama.cpp или Ollama
Модель	`Qwen/Qwen3-4B-Instruct-2507`, FP16	GGUF той же модели, квантование Q4_K_M
Контекст	32 768 токенов	16 384 токенов
Параллельных запросов	сколько поместится в KV-кеш	1 слот в экспериментах 1–2, 4 слота в эксперименте 3

Можно выбрать другую открытую instruct-модель на 3–9 млрд параметров, если она поддерживает вызов инструментов (понадобится с ЛР4), помещается в память вместе с кешем нужной длины и её конфигурация известна. У модели этой работы для llama.cpp нет официальных GGUF, но есть популярные сборки сообщества, например `bartowski/Qwen_Qwen3-4B-Instruct-2507-GGUF`; запишите, какую вы взяли, и её хеш.

Паспорт сервера: модель, ревизия и формат весов; сервер и его версия; команда запуска со всеми флагами; предел контекста; ёмкость KV-кеша из журнала запуска (у vLLM — строка `GPU KV cache size: … tokens, Maximum concurrency for … tokens per request: …x`); устройство, память, число потоков CPU.

Команды запуска для Colab — в приложении. Для последующих работ сохраните в репозитории `endpoint.yaml`: адрес сервера, имя модели и параметры генерации по умолчанию.

3.2. Клиент-измеритель

Клиент — адаптер каркаса ЛР0: принимает список сообщений и параметры, отправляет запрос с `stream=True` и `stream_options={"include_usage": true}`, отмечает t_0, t_1, время каждого фрагмента и t_N, берёт число токенов из `usage` и возвращает запись для `results.jsonl`: id запроса, условие опыта, входные и выходные токены, TTFT, TPOT, полное время, причину завершения и статус. Сводки и графики строятся из этих записей и сохраняются в MLflow («Правила», п. 4).

Чтобы ответ был ровно заданной длины, генерацию нельзя обрывать на токене конца текста: у vLLM для этого есть параметр `ignore_eos` (и `min_tokens`), у llama.cpp — `ignore_eos`. Их передают через `extra_body` клиента `openai`. Проверяйте фактическую длину по `usage.completion_tokens`.

3.3. Запросы заданной длины

Возьмите длинный текст из общественного достояния — «Pride and Prejudice» с Project Gutenberg [14] — и сохраните файл и его SHA-256 в манифест данных. Запрос строится так: уникальная метка (например, номер запроса), инструкция «Summarize the following text in one sentence.» и начало текста нужной длины в токенах, отрезанное токенизатором модели. Длину итогового запроса после шаблона диалога проверяйте по `usage.prompt_tokens`. Уникальная метка стоит в самом начале: тогда кеширование префикса не может переиспользовать предыдущие запросы.

3.4. Проверки

- Арифметика метрик на заданных отметках: $t_0 = 0$, $t_1 = 0{,}4$ с, $t_N = 2{,}4$ с, $N = 101$ дают TTFT 400 мс и TPOT 20 мс.
- Запрос с `max_tokens = 64` и `ignore_eos` возвращает ровно 64 токена.
- Один и тот же запрос при температуре 0, отправленный дважды подряд, — сравните тексты и запишите, совпали ли они.
- Ёмкость KV-кеша из журнала сервера, умноженная на 144 КБ на токен, примерно равна строке `Available KV cache memory` того же журнала. Если нет, разберитесь, почему (другая модель, другой тип данных кеша).

4. Эксперименты

До запусков запишите прогноз: как TTFT зависит от длины запроса — линейно или быстрее; какой TPOT вы ожидаете, исходя из оценки п. 2.1; при какой нагрузке сервер перестанет укладываться в пределы.

Во всех опытах температура 0; перед каждой серией — один прогревочный запрос, в статистику он не входит («Правила», п. 7).

4.1. Эксперимент 1. Длина запроса и TTFT

1. Запросы длиной 256, 1024, 4096, 8192 и 16 384 токенов на GPU либо 128, 512, 1024, 2048 и 4096 на CPU; по 5 запросов на длину, ответ — 16 токенов, запросы отправляются по одному. Для этой серии отключите кеширование префикса на сервере (у vLLM — `--no-enable-prefix-caching`) или убедитесь, что уникальная метка делает его бесполезным.
2. График TTFT от длины запроса: медиана и диапазон по 5 запросам. Посчитайте скорость prefill в токенах в секунду для каждой длины. Остаётся ли она постоянной и почему может падать на длинных запросах?
3. **Кеширование префикса.** Включите его и трижды отправьте один и тот же запрос на 8192 токена (на CPU — на 2048). Во сколько раз изменился TTFT со второго раза? Объясните, чем это поможет агенту из ЛР4.

4.2. Эксперимент 2. Длина ответа и скорость генерации

Запрос на 1024 токена, ответ — 16, 64, 256 и 1024 токенов с `ignore_eos`, по 5 запросов на длину. Постройте график полного времени от длины ответа и аппроксимируйте его прямой: свободный

член должен быть близок к TTFT, наклон — к TPOT. Сравните TPOT с нижней границей из п. 2.1 и объясните разницу.

4.3. Эксперимент 3. Нагрузка

1. До опыта запишите пределы для интерактивного ассистента, например: TTFT у 90% запросов не больше 2 с и медиана TPOT не больше 100 мс на GPU. На CPU выберите пределы сами и обоснуйте.
2. Уровни нагрузки — 1, 2, 4, 8, 16 и 32 одновременных клиента на GPU либо 1, 2 и 4 на CPU. Каждый клиент отправляет следующий запрос, как только получил ответ на предыдущий. На уровне — $\max(16,\ 4c)$ запросов, где c — число клиентов; запросы по 1024 токена с разным текстом, ответ — 128 токенов.
3. Для каждого уровня: пропускная способность (выходные токены в секунду), запросы в секунду, медиана и 90-й процентиль TTFT, медиана TPOT, goodput. У vLLM снимайте метрики `vllm:num_requests_waiting` и `vllm:kv_cache_usage_perc` с адреса `/metrics` до и после каждого уровня [15].
4. График «пропускная способность — задержка» и таблица уровней. Найдите наибольшую нагрузку, при которой пределы выполняются. Сопоставьте момент, когда запросы начали ждать в очереди, с ёмкостью KV-кеша: сколько запросов по 1024 + 128 токенов в неё помещается?

4.4. По желанию

- Сравните свой измеритель с `vllm bench serve` на той же нагрузке [15].
- Квантованная модель — например, AWQ или GGUF Q4 против FP16: память, TTFT, TPOT, пропускная способность.
- llama.cpp и vLLM на одной и той же T4.

5. Что сдать

- Паспорт сервера, команды запуска и журналы, `endpoint.yaml`.
- Код клиента и генератора запросов; манифест данных с текстом и хешем.
- Артефакты трёх экспериментов и графики из MLflow.
- Отчёт на 3–4 страницы: прогноз; расчёт памяти весов и KV-кеша; три графика и таблица нагрузки; вывод о пределах и о том, сколько пользователей выдерживает ваш сервер; что это значит для ЛР2–ЛР8; кодогенерация.

Ошибки постановки здесь чаще всего такие: TTFT измерен с включённым кешированием префикса на повторяющихся запросах; длина ответа не проверена по `usage`; температура не передана явно; в статистику попал прогрев.

Вопросы к защите

1. Почему TTFT растёт с длиной запроса, а TPOT от длины ответа почти не зависит? Покажите на своих графиках.
2. Пересчитайте TPOT по своим отметкам времени. Почему в знаменателе $N - 1$, а не N?
3. Сколько памяти займёт KV-кеш вашей модели для 8 пользователей с запросами по 4000 токенов? Поместится ли это на вашем оборудовании?
4. Почему суммарная пропускная способность растёт с числом клиентов, а время ответа каждому ухудшается?
5. Что показали повторные запросы с кешированием префикса? Почему это особенно важно для агента с 24 инструментами?
6. Одинаковый запрос при температуре 0 дал разные ответы. Это ошибка? Откуда это берётся?
7. Уменьшите предел контекста сервера вдвое. Предскажите, как изменятся ёмкость KV-кеша и наибольшая допустимая нагрузка, и проверьте.

Источники

1. Hugging Face. LLM Course: How do Transformers work? — токены, трансформер, генерация текста; для этой работы достаточно глав 1.4–1.6.
2. Databricks. LLM Inference Performance Engineering: Best Practices — TTFT, TPOT, пропускная способность; почему prefill упирается в вычисления, а decode — в память.
3. Pope R. et al. Efficiently Scaling Transformer Inference. MLSys, 2023. Статья — модель затрат prefill и decode и память KV-кеша (разд. 2–3).
4. NVIDIA. Tesla T4 datasheet — 16 ГБ памяти, 300 ГБ/с.
5. Hugging Face Transformers. Caching — что хранит KV-кеш и зачем.
6. Конфигурация Qwen3-4B-Instruct-2507 и карточка модели.
7. Zhong Y. et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI, 2024. Статья — пределы TTFT и TPOT и goodput (разд. 2); краткое изложение.
8. OpenAI. API Reference: Chat Completions.
9. Kwon W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023. Статья — KV-кеш блоками и объединение запросов в vLLM (разд. 3–4).
10. vLLM. `vllm serve`, OpenAI-совместимый сервер и дополнительные параметры, вызов инструментов.
11. llama.cpp. llama-server.
12. Ollama. OpenAI compatibility, Context length.
13. vLLM. Reproducibility — почему ответы при температуре 0 могут различаться.
14. Austen J. Pride and Prejudice. Project Gutenberg, № 1342.
15. vLLM. Метрики `/metrics`, `vllm bench serve`.

Приложение. Запуск и измерение

vLLM в Colab (GPU T4):

```
pip install vllm
nohup vllm serve Qwen/Qwen3-4B-Instruct-2507 --dtype half --max-model-len 32768 \
    --enable-auto-tool-choice --tool-call-parser hermes --port 8000 > vllm.log 2>&1 &
# дождитесь, пока curl -s localhost:8000/v1/models начнёт отвечать, затем:
grep "KV cache" vllm.log
```

Для эксперимента 1 запустите второй экземпляр с флагом `--no-enable-prefix-caching` (первый предварительно остановите: на T4 два сервера не поместятся).

llama.cpp (CPU):

```
llama-server -m Qwen_Qwen3-4B-Instruct-2507-Q4_K_M.gguf -c 16384 -np 4 --port 8080
```

Шаблон диалога с поддержкой инструментов (`--jinja`) у современных сборок включён по умолчанию. Проверьте в журнале запуска, сколько контекста получает один слот: в зависимости от версии и флага `--kv-unified` предел `-c` либо общий для всех слотов, либо делится между ними поровну. Во втором случае при `-np 4` запросу достанется только четверть, поэтому эксперименты 1–2 запускайте с `-np 1`.

Потоковый запрос с замером времени:

```
import time
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="none")

def timed_chat(messages, max_tokens, **params):
    t0 = time.perf_counter()
    stream = client.chat.completions.create(
        model="Qwen/Qwen3-4B-Instruct-2507", messages=messages, max_tokens=max_tokens,
```

```
        temperature=0, stream=True, stream_options={"include_usage": True},
        extra_body={"ignore_eos": True}, **params)
    t1 = t_last = None
    usage, text = None, []
    for chunk in stream:
        if chunk.usage is not None:
            usage = chunk.usage                          # последний фрагмент — только usage
        if chunk.choices and chunk.choices[0].delta.content:
            now = time.perf_counter()
            t1 = t1 or now
            t_last = now
            text.append(chunk.choices[0].delta.content)
    n = usage.completion_tokens
    return {"ttft_s": t1 - t0, "tpot_s": (t_last - t1) / (n - 1) if n > 1 else None,
            "e2e_s": t_last - t0, "prompt_tokens": usage.prompt_tokens, "output_tokens": n,
            "text": "".join(text)}
```

Сервер может присылать в одном фрагменте несколько токенов, поэтому TPOT считается по числу токенов из `usage`, а не по числу фрагментов.

Нагрузка — `asyncio` и `AsyncOpenAI`: *c* задач, каждая в цикле берёт следующий запрос из общей очереди, пока та не опустеет. Пропускная способность уровня — сумма выходных токенов, делённая на время от первого отправленного до последнего полученного ответа.